Test Planning

A **test plan** is a document that explains **what we will test, why we will test, how we will test, who will test, and when testing will be completed**.

It helps the QA team and stakeholders understand the full testing process before starting the test project.

Test planning includes:

1. Testing Context

This explains the basic background of testing.

It includes:

- What will be tested
- What will not be tested
- Test objectives
- Project limitations
- Requirement documents or test basis

2. Assumptions and Constraints

This explains what we are assuming and what limitations we have.

Example:

- The test environment will be ready on time
- Developers will fix critical bugs quickly
- Testing must be completed within 2 weeks
- Limited testers or limited devices are available

3. Stakeholders

This explains who is involved in the testing project and what their responsibilities are.

Example:

- QA team
- Developers

- Project manager
- Product owner
- Business team
- Client

It may also include whether new testers need to be hired or trained.

4. Communication

This explains how the team will communicate during testing.

Example:

- Daily standup meeting
- Bug reporting process
- Test summary report
- Email or Slack updates
- Weekly QA status meeting

5. Risk Register

This lists possible risks that can affect the product or the testing process.

Example:

- Important feature may fail
- Test environment may not be stable
- Requirement may change frequently
- Testing time may be too short
- Critical bugs may be found near release

6. Test Approach

This explains how testing will be performed.

It includes:

- Test levels, such as unit, integration, system, and acceptance testing
- Test types, such as functional, regression, performance, security, and usability testing
- Test techniques, such as boundary value analysis, equivalence partitioning, and exploratory testing
- Test deliverables, such as test cases, bug reports, and test summary reports

- Entry criteria and exit criteria
- Test data requirements
- Test environment requirements
- Metrics to be collected
- Level of tester independence

7. Resource, Budget, and Schedule

This explains what resources are needed and when testing activities will happen.

It includes:

- Number of testers needed
- Required tools
- Required devices
- Required budget
- Testing timeline
- Test execution schedule
- Release deadline

In simple words:

A test plan answers these questions:

- What will we test?
- Why will we test?
- How will we test?
- Who will test?
- When will testing start and finish?
- What risks may affect testing?
- What resources are needed?
- What conditions must be met before release?

So, a test plan works like a roadmap for the complete testing process.

Test Strategy

A **test strategy** is a high-level guideline that explains **how testing will be performed across a project, product, or organization**.

It defines the overall testing direction, approach, standards, tools, responsibilities, risk management, and quality expectations.

Example:

For an eCommerce project, the test strategy may include:

- Performing API testing first, then UI testing
- Using Playwright for automation
- Giving high priority to payment, login, and order flows
- Running performance testing before release

In simple terms:

Test Strategy = The big-picture approach for how testing will be done

Test Effort Estimation

1. Estimation Based on Ratios

This technique uses data from previous similar projects to estimate testing effort.

Example:

In previous projects, your company found that:

Development effort : Testing effort = 3 : 2

Now, if the current project development effort is **600 person-days**, then testing effort can be estimated as:

Testing effort = 400 person-days

So, if development takes 3 parts, testing usually takes 2 parts.

2. Extrapolation

This technique estimates future testing effort based on the progress of the current project.

Example:

Suppose in the first sprint, QA tested **5 features in 10 days**.

Now, the project has **20 similar features**.

So, estimated testing effort:

5 features = 10 days

20 features = 40 days

This is useful when the team starts testing and then predicts the remaining effort based on early progress.

3. Wideband Delphi

In this technique, multiple experts estimate individually. Then they discuss differences and agree on a final estimate.

Example:

For testing a payment feature:

- Tester 1 estimates: 3 days
- Tester 2 estimates: 5 days
- Tester 3 estimates: 4 days

They discuss the assumptions, risks, and complexity. Finally, they agree:

Final estimate = 4 days

This is useful when expert judgment is needed.

4. Three-Point Estimation

This technique uses three estimates:

- **Optimistic:** Best-case estimate

- **Most likely:** Normal estimate
- **Pessimistic:** Worst-case estimate

Formula:

E = (Optimistic + 4 × Most likely + Pessimistic) / 6

Example:

For testing checkout functionality:

- Optimistic estimate = 2 days
- Most likely estimate = 4 days
- Pessimistic estimate = 8 days

So,

E = (2 + 4×4 + 8) / 6

E = (2 + 16 + 8) / 6

E = 26 / 6

E = 4.3 days

So, the estimated testing effort is around **4 to 5 days**.

In simple terms

- **Ratio-based estimation:** Estimate using past project ratios.
- **Extrapolation:** Estimate based on current progress.
- **Wideband Delphi:** Experts discuss and agree on an estimate.
- **Three-point estimation:** Estimate using best-case, normal-case, and worst-case values.

Acceptance Criteria

Acceptance criteria (AC) are the conditions that a software product must meet to be accepted by a user, a customer, or other systems. They are unique for each user story and define the feature behavior from the end-user's perspective.

2 ways can be written:

1. Scenario oriented
2. Rule oriented

Scenario Oriented:

- It is approached through the Given/When/Then (GWT) sequence that looks like this:
    - Given some precondition
    - When I do some action
    - Then I expect some result
- This approach was inherited from behavior-driven development (BDD) and provides a consistent structure that helps testers define when to begin and end testing a particular feature.

Scenario Oriented: Example

Feature: User should Login to access the portal

As a new user

I want to log in to the website

So that the user can login successfully

Scenario #1: Successful Log in to the website

Given A user visit the website

When the user clicks Sign-in

And enters a valid email <email> and password <password>

And clicks Sign-in/submit button

Then the user should be successfully logged into the site

Scenario #2: Unsuccessful Log in to the website

Given A user visit the website

When the user enters an invalid email <email> and password <password>

And clicks Sign-in/submit button

Then the user should get a warning message

Rule oriented: Example

This type of acceptance criteria objectives are same as the scenario oriented. But the writing styles are different. It is written by bullet point verification list or tabulated form of input output mapping.

For example regarding a send money transaction:

1. Sender should have enough balance
2. After successful transaction sender balance should be deducted with corresponding service fee
3. Sender and receiver should get SMS
4. Sender transaction list should be updated.